Task 2 Report

Submitted by Haseeb Ijaz

Game Link: https://ai.studio/apps/3bf780e5-fb6d-44d3-a5fc-5b2d8a48b7cd

o What prompts were used

I asked chatgpt to generate me a Prompt which I gave to the Google AI Studio and it then generated a game.

o Which prompts worked effectively

I prefer to ask it in the long prompt format. My first prompt was very long.

o Which prompts did not work well

No idea

o How the prompts were refined

By using Chatgpt to refine them..

o The complete process followed to achieve the final result

I asked chatgpt to generate me a Prompt which I gave to the Google AI Studio and it then generated a game.

o Which AI tools/models were used

- Google AI Studio
- ChatGPT

Initial Prompt

Modern Retro Snake Game Prompt

Create a complete modern retro Snake game as a deployable web app.

1. Project Goal

Build a Snake game inspired by classic retro arcade games, but redesigned with a modern, minimalist aesthetic and smooth animations.

2. Feel

The game should feel nostalgic.

3. Technology Stack

Do not use a backend, database, or authentication system. The app should be fully client-side.

- Frontend Framework: React
- Language: TypeScript
- Build Tool: Vite
- Styling: Tailwind CSS
- Game Rendering: HTML5 Canvas
- State Management: React hooks only, no external state library
- Persistence: LocalStorage for high scores and settings
- Deployment: Static hosting compatible with Vercel, Netlify, GitHub Pages, or Cloudflare Pages

4. Design Direction

Design Style: Retro arcade meets modern glassmorphism UI

5. Visual Style

- Dark futuristic background
- Neon accent colors such as green, cyan, purple, and pink
- Pixel-inspired typography for headings
- Smooth modern UI panels with rounded corners
- Subtle glow effects around the snake, food, score, and buttons
- Grid-based game board with a faint retro scanline or arcade-screen effect
- Animated start screen and game-over screen
- Desktop-focused layout

6. UI Layout

Include:

- Main game canvas centered on the page
- Score panel
- High score display
- Current level or speed indicator
- Start / Pause / Restart buttons
- Game mode selector

- Settings panel
- Keyboard control hints

The interface should feel like a premium browser game, not a plain demo.

7. Core Features

Implement the following features:

Required Features

- Classic Snake gameplay
- Start screen
- Pause and resume
- Restart game
- Game over screen
- Current score
- Persistent high score using LocalStorage
- Increasing difficulty as score increases
- Smooth keyboard controls
- Sound toggle
- Theme toggle
- Different food types:
    - Normal food
    - Bonus food with timeout
    - Slow-down food
- Simple particle effect when food is eaten
- Screen shake on collision
- Countdown before game starts
- Achievement-style messages such as:
    - “Speed Up!”
    - “New High Score!”

8. Game Mechanics

Board

- Use a fixed grid system.
- Recommended board size: 20 x 20.
- Each snake and food segment should align to the grid.
- Canvas should render crisply at a desktop-friendly size while preserving grid logic.

Snake Movement

- Snake moves one grid cell per tick.
- Direction controls:

    - Arrow keys
    - WASD keys
- Prevent instant reversal into itself.

Food

Food should spawn randomly on empty cells.

Eating food should:

- Increase snake length
- Increase score
- Trigger visual feedback
- Possibly increase speed after certain score thresholds

Scoring

Use this scoring model:

- Normal food: +10 points
- Bonus food: +30 points
- Speed increase every 50 points
- Save highest score in LocalStorage

Collisions

Handle collisions with:

- Walls
- Snake body

For No Walls Mode, the snake should wrap around the board edges instead of dying.

Difficulty

- Initial speed should be beginner-friendly.
- Speed should gradually increase as the player scores.
- The game should remain playable and not become instantly impossible.

9. Screens

Create the following screens or states:

Start Screen

Include:

- Game title
- Short tagline
- Start button
- Game mode selector
- Controls explanation

Playing Screen

Include:

- Game canvas
- Score
- High score
- Pause button
- Current mode
- Current speed or level

Paused Screen

Include:

- “Paused” overlay
- Resume button
- Restart button

Game Over Screen

Include:

- Final score
- High score
- “New High Score” message when applicable
- Restart button
- Return to menu button

10. Controls

Desktop Controls

- Arrow keys to move
- WASD to move
- Spacebar to pause/resume
- Enter to start/restart

11. Deployment Requirements

The project must be deployable as a static site.

Include support for:

codeBash

npm install

npm run dev

npm run build

npm run preview

The final app should work after running:

codeBash

npm install

npm run build

It should be deployable directly to:

- Vercel
- Netlify
- GitHub Pages
- Cloudflare Pages

Do not require a backend s